

串与模式匹配

失配之后，怎样复用已经确认的前缀信息

408 · 数据结构 Topic DS-03

母问题：失配之后，哪些信息真的已经知道？

给定文本 T 和模式 P ，匹配不是“把两个字符串碰运气比较”，而是一个不断维护匹配状态的扫描过程：

Matched Prefix → Mismatch → Reusable Border
→ Next Alignment → Continue / Accept.

朴素算法在失配后把模式移回下一个起点，因此反复比较文本；KMP 观察已匹配的模式前缀，寻找同时也是后缀的最长边界（border），只缩短模式状态。箭头表示一次匹配状态机的时间顺序，不是某个特定 'next' 数组下标的背诵顺序。

本册定位与 Owner 边界

- **Owns**：串的顺序对象、朴素匹配、prefix function/failure 信息、失配回退与 KMP 主循环。
- **Uses**：Atlas 的表示—操作—不变量—成本语言；底层字符序列只借用一般线性存储，不重新定义 DS01。
- **Outputs**：向算法专题提供线性时间模式匹配的可调用接口。
- **Stops**：完整 DFA/自动机最小化、多模式匹配与文本索引属于 Extension 或其他专题。

目录

1	对象、契约与第一基线	3
1.1	串 ADT、子串数量与存储边界	3
1.2	朴素 BF 的两个指针与回退公式	3
2	Prefix Function：模式自己的可复用结构	3
2.1	把 π 真正算一遍：ababaca	4
3	KMP 主循环：模式状态回退，主串不倒退	4
3.1	KMP 的线性时间证明：把回退次数当作势能	5
3.2	next、nextval 与“首个匹配”之外的输出契约	5
4	代码：把两种扫描策略放在同一契约下	5
4.1	朴素基线：每个起点重新建立 matched	5
4.2	Prefix function：失配时只沿 border 链回退	6
4.3	KMP：失配不倒退主串	6
4.4	测试：重复前后缀与边界返回值	7

5	复杂度与边界	8
5.1	从单模式扫描到全文搜索：索引改变了成本归属	8
6	概念边界与做题控制协议	9
7	全科坐标与来源处理	9

1 对象、契约与第一基线

串是一个有序字符序列；模式匹配的输出契约是：返回 P 在 T 中第一次出现的起始位置，若不存在则返回 ‘npos’。本册代码另约定空模式返回 0，这使边界行为显式而不依赖调用者猜测。

朴素匹配枚举起点 $s = 0, 1, \dots, |T| - |P|$ ，在每个起点从左到右比较。若在第 j 个字符失配，已经比较过的前缀信息只用于判定当前起点失败，算法不会把它带到下一起点。最坏比较次数可以达到 $\Theta((n-m+1)m)$ ，其中 $n = |T|, m = |P|$ 。

为什么需要新机制

文本 $T = \text{aaaaab}$ 、模式 $P = \text{aaab}$ 时，许多起点都会先匹配若干个 ‘a’ 再失配。朴素算法每次都重新确认这些 ‘a’；但“已经看到的后缀由若干 ‘a’ 构成”并没有消失。若模式自身存在重复前后缀，就可以把这份信息压缩成下一次应比较的模式位置。

1.1 串 ADT、子串数量与存储边界

串是字符受限的线性表，但题面中的“字符个数”“存储长度”“子串长度”仍需分别解释。长度为 n 的串有 $n(n+1)/2$ 个非空连续子串（按起点和终点确定），若把空串也算入则再加 1；相同内容出现在不同位置时，在“子串个数”问题中是否去重必须由题设声明。串的连接、截取、比较和模式匹配都依赖字符顺序，但不要求底层一定使用 C 风格的 ‘0’ 终止符。

顺序串可以用固定数组、动态数组或长度字段加字符区表示；链式串可以把字符分块存储。表示改变的是随机访问、连接、扩容和额外空闲字符的成本，不改变子串的逻辑定义。若题目给出“长度不计终止符”，‘0’ 只是实现边界；若问存储容量，则必须把终止符、块内空闲字符或长度字段的约定单独计入。

1.2 朴素 BF 的两个指针与回退公式

朴素匹配常用文本指针 i 和模式指针 j 。当 $T[i] = P[j]$ 时同时前进；失配且 $j > 0$ 时，当前起点已经失败，模式回到 0，文本应回到“该起点后的下一个字符”：

$$i \leftarrow i - j + 1, \quad j \leftarrow 0.$$

如果代码采用“比较失败后循环末尾再统一执行 $i++$ ”，就会把同一语义写成 $i \leftarrow i - j + 2$ 。两个公式不是不同算法，而是指针更新时机不同；判断是否正确要看下一轮比较的文本位置，而不是死记加一的数值。

BF 的不变量是：进入每个起点时， i 指向该起点， $j = 0$ ；循环结束时，已经比较的 $T[s..i-1]$ 与 $P[0..j-1]$ 相等，或已找到第一个不等字符。它的最坏界来自很多起点重复确认同一文本片段，而不是来自“回退”这个词本身。用 BF 作为基线，可以明确 KMP 真正删除的是“重新建立匹配前缀”的重复工作。

2 Prefix Function：模式自己的可复用结构

对模式前缀 $P[0..i]$ ，定义 $\pi[i]$ 为其最长真前缀且同时为后缀的长度。它只由模式自身决定，不依赖文本：

$$\pi[i] = \max\{k < i + 1 : P[0..k-1] = P[i-k+1..i]\}.$$

若当前边界长度为 $b = \pi[i - 1]$ 且新字符不匹配，任何更短的候选边界都必然是长度 b 边界的边界，因此可令 $b \leftarrow \pi[b - 1]$ ，继续尝试，而不用从 0 重新比较。

这给出构造不变量：处理 $P[0..i]$ 后，‘ $\text{prefix}[i]$ ’ 恰是该前缀的最长 border；回退链严格变短，所以整个构造是线性的摊还过程。

2.1 把 π 真正算一遍：ababaca

逐个加入新字符，并始终问 “当前最长 border 能否延长；不能时，它自己的最长 border 是谁”：

i	$P[i]$	旧候选 b	比较、回退与可复用事实	$\pi[i]$
0	a	无	单字符没有非空真 border	0
1	b	0	$b \neq a$ ，无法延长	0
2	a	0	$a = a$ ，空 border 延长为 ‘a’	1
3	b	1	$b = P[1]$ ，‘a’ 延长为 ‘ab’	2
4	a	2	$a = P[2]$ ，‘ab’ 延长为 ‘aba’	3
5	c	3	$c \neq P[3]$ ： $3 \rightarrow 1 \rightarrow 0$ ，仍不等	0
6	a	0	$a = P[0]$ ，重新得到 ‘a’	1

因此 $\pi = [0, 0, 1, 2, 3, 0, 1]$ 。第 5 项按 $3 \rightarrow 1 \rightarrow 0$ 回退，是因为长度 3 的 border 已失败，剩余候选只能是这个 border 自己的 border；其他长度不可能同时保持前缀和后缀。

3 KMP 主循环：模式状态回退，主串不倒退

扫描文本下标 i 时，变量 ‘ matched ’ 表示当前文本后缀与模式前缀相等的长度。失配时反复执行

$$\text{matched} \leftarrow \pi[\text{matched} - 1]$$

直到匹配、归零或接受完整模式；文本下标 i 仍只前进一次。若 ‘ $\text{matched} = m$ ’，起点为 $i + 1 - m$ 。

一次完整失配：文本不回头，模式状态缩短

用 $P = \text{ababaca}$ 扫描文本片段 abababaca 。读完 ababa 后， $\text{matched} = 5$ ；下一文本字符是 ‘b’，而 $P[5] = c$ 。已知文本后缀 aba 等于模式前缀 aba ，所以令

$$5 \rightarrow \pi[4] = 3.$$

仍用同一个文本字符 ‘b’ 比较 $P[3]$ ，匹配后变成 $\text{matched} = 4$ ；主串下标没有倒退，也没有重新比较已经证实的 aba 。这就是代码中 ‘while’ 回退后紧接 ‘if’ 比较的语义。

状态	迁移	必须保持的事实
$\text{matched} = 0$	比较当前文本字符与 $P[0]$	没有可复用的模式前缀
$\text{matched} > 0$ 且失配	沿 prefix 链缩短模式状态	不移动文本下标，保留已证实的后缀
匹配后	$\text{matched}++$	当前状态代表长度为 matched 的相等前缀
$\text{matched} = m$	返回 $i + 1 - m$	所有模式字符已连续匹配

3.1 KMP 的线性时间证明：把回退次数当作势能

主串下标 i 从不减小；模式状态 *matched* 每次成功比较最多增加 1，每次失配回退都严格跳到更短的 border。一次回退可能跨越多级 prefix 链，但它消耗了此前积累的 *matched* 长度，因此在整个扫描中回退总次数不超过成功增长次数的常数倍。用势能 $\Phi = \text{matched}$ 表示，成功比较增加势能，回退减少势能，故 n 个文本字符的摊还比较次数为 $O(n)$ ，再加 $O(m)$ 的 prefix 构造得到 $O(n + m)$ 。这比只写“while 很快”更能解释最坏周期串也不会退化。

3.2 next、nextval 与“首个匹配”之外的输出契约

不同教材的‘next’可能把状态编码为“已匹配长度”、失配字符下标或带 -1 哨兵的状态；它们的共同语义都是“失配后仍可能成立的最长 border”。因此转换时先统一为长度数组 π ，再按目标下标重写初始化和回退语句。

若模式的下一个字符与回退后要比较的字符相同，普通‘next’会先跳到一个必然再次失配的状态；‘nextval’继续沿 failure 链跳过这些等字符状态。它可能减少重复比较，尤其是高度周期化的模式，但不改变正确性或最坏 $O(n + m)$ 上界。‘nextval’是对状态机的优化编码，不应与 prefix 定义混为另一套数学对象。

匹配接口还要区分“返回首个位置”和“枚举全部出现”。若允许重叠匹配，成功一次后不能把 *matched* 清零，而应令 $\text{matched} \leftarrow \pi[\text{matched} - 1]$ ，继续扫描下一个文本字符；例如模式‘aba’在‘ababa’中出现于 0 和 2。若只返回首个位置，接受状态即可立即结束。接口契约改变，状态复位策略随之改变。

4 代码：把两种扫描策略放在同一契约下

完整 C++17 实现位于 code/ds03_string_matching.hpp，测试位于 code/ds03_string_matching_test.cpp；正文代码块直接从真实源文件抽取。

4.1 朴素基线：每个起点重新建立 matched

```
1 inline std::size_t naive_search(std::string_view text, std::string_view pattern
2     ) {
3     if (pattern.empty()) {
4         return 0;
5     }
6     if (pattern.size() > text.size()) {
7         return std::string_view::npos;
8     }
9     for (std::size_t start = 0; start + pattern.size() <= text.size(); ++start) {
10         std::size_t matched = 0;
11         while (matched < pattern.size() && text[start + matched] == pattern[matched]) {
12             ++matched;
13         }
14         if (matched == pattern.size()) {
15             return start;
16         }
17     }
18     return std::string_view::npos;
```

代码清单 1: 朴素匹配的起点枚举与局部比较

朴素版本的重要性在于提供复杂度基线：它没有错误，只是没有利用跨起点的结构信息。任何声称 KMP 更快的解释，都应能指出它省掉了哪类重复比较。

4.2 Prefix function: 失配时只沿 border 链回退

```

1  std::vector<int> prefix(pattern.size(), 0);
2  for (std::size_t i = 1; i < pattern.size(); ++i) {
3      int border = prefix[i - 1];
4      while (border > 0 && pattern[i] != pattern[static_cast<std::size_t>(border)
5          ]) {
6          border = prefix[static_cast<std::size_t>(border) - 1];
7      }
8      if (pattern[i] == pattern[static_cast<std::size_t>(border)]) {
9          ++border;
10     }
11     prefix[i] = border;
12 }
13 return prefix;
14 }
15 inline std::size_t kmp_search(std::string_view text, std::string_view pattern)
16 {
17     if (pattern.empty()) {
18         return 0;
19     }

```

代码清单 2: prefix function 的回退链构造

‘border’ 只表示当前候选前缀长度。每次回退都跳到更短 border，因此不会形成无限循环；若新字符与候选前缀末字符相等，再增长一次。prefix 数组是 KMP 运行阶段可复用信息的唯一来源。

4.3 KMP: 失配不倒退主串

```

1  std::size_t matched = 0;
2  for (std::size_t i = 0; i < text.size(); ++i) {
3      while (matched > 0 && text[i] != pattern[matched]) {
4          matched = static_cast<std::size_t>(prefix[matched - 1]);
5      }
6      if (text[i] == pattern[matched]) {
7          ++matched;
8      }
9      if (matched == pattern.size()) {
10         return i + 1 - pattern.size();
11     }
12 }
13 return std::string_view::npos;
14 }
15

```

```

16 } // namespace ipara::ds03
17
18 #endif

```

代码清单 3: KMP 主循环与完整模式接受

主串循环变量 ‘i’ 单调递增；模式状态 ‘matched’ 才可能沿 prefix 链下降。于是预处理 $O(m)$ 加扫描 $O(n)$ ，总时间 $O(n + m)$ ，额外空间 $O(m)$ 。这不是 “next 公式自动带来线性”，而是由两个单调性共同保证。

4.4 测试：重复前后缀与边界返回值

```

1 using ipara::ds03::naive_search;
2 using ipara::ds03::prefix_function;
3
4 void test_empty_and_boundary_patterns() {
5     assert(naive_search("abc", "") == 0);
6     assert(kmp_search("abc", "") == 0);
7     assert(naive_search("abc", "abcd") == std::string_view::npos);
8     assert(kmp_search("abc", "abcd") == std::string_view::npos);
9     assert(naive_search("", "a") == std::string_view::npos);
10    assert(kmp_search("", "a") == std::string_view::npos);
11 }
12
13 void test_naive_and_kmp_agree() {
14     const std::vector<std::pair<std::string_view, std::string_view>> cases = {
15         {"abcabcabcd", "abcabcd"}, {"aaaaab", "aaab"}, {"mississippi", "issi"},
16         {"abcdef", "gh"}, {"abababab", "ababa"};
17     for (const auto& [text, pattern] : cases) {
18         assert(naive_search(text, pattern) == kmp_search(text, pattern));
19     }
20 }
21
22 void test_prefix_reuse_information() {
23     assert((prefix_function("ababaca") == std::vector<int>{0, 0, 1, 2, 3, 0, 1}));
24     assert((prefix_function("aaaa") == std::vector<int>{0, 1, 2, 3}));
25     assert((prefix_function("aabaaab") == std::vector<int>{0, 1, 0, 1, 2, 2, 3}));
26 }
27
28 void test_repeated_mismatch_does_not_lose_text_progress() {
29     assert(kmp_search("ababababca", "abababca") == 2);
30     assert(kmp_search("aaaaaaaaab", "aaab") == 6);
31 }
32
33 std::vector<std::string> binary_strings(std::size_t maximum_length) {
34     std::vector<std::string> strings{" "};
35     for (std::size_t length = 1; length <= maximum_length; ++length) {
36         const std::size_t count = std::size_t{1} << length;
37         for (std::size_t mask = 0; mask < count; ++mask) {

```

代码清单 4: 空模式、重复前后缀、失配回退与基线一致性

测试要求朴素与 KMP 在多组文本/模式上返回同一首个位置，直接检查 `prefix` 数组的重复结构，并覆盖空模式、模式过长、完全无匹配和失配后仍有答案的情形。

测试还枚举二字母表上长度不超过 6 的全部文本、长度不超过 4 的全部模式，逐对验证 KMP 与朴素算法返回同一首个位置。穷举不是正确性证明，但会系统攻击空模式、重叠匹配、周期串、长 `border` 链与无匹配状态。

```
1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \\  
2   code/ds03_string_matching_test.cpp -o /tmp/ds03_test  
3 /tmp/ds03_test
```

5 复杂度与边界

任务	朴素匹配	KMP	关键前提	失败边界
预处理模式	无	$O(m)$	<code>prefix</code> 语义固定	空模式
扫描文本	$O((n - m + 1)m)$ 最坏	$O(n)$	主串下标单调前进	$m > n$
总额外空间	$O(1)$	$O(m)$	只保存模式信息	大模式内存预算
返回位置	首个合法起点	首个合法起点	接受后立即返回	无匹配返回 <code>npos</code>

不同教材的 ‘`next`’ 可能使用 “失配时跳到的下标” 或 “最长 `border` 长度” 两种编码。做题时先写语义，再把题目下标转换到该语义；不能拿另一版本的初值和边界直接替换。

从长度语义翻译到教材 `next`

本册只维护 $\pi[i] = \text{“}P[0..i] \text{的最长 border 长度”}$ 。若教材把 ‘`next[j]`’ 定义为 “模式状态 j 失配后跳到的状态”，零基长度状态下对应 $\pi[j - 1]$ ($j > 0$)。若教材使用 `-1` 哨兵或一基下标，先画出 “已经匹配多少个字符”，再平移下标；数组首项长什么样不是算法本体。

5.1 从单模式扫描到全文搜索：索引改变了成本归属

对多篇文档做查询时，逐篇运行 BF 或 KMP 的成本为 “文档总字符数乘以查询次数”。KMP 只减少单篇扫描的失配浪费，并没有把文档集合变成可按词定位的结构。若查询重复出现，可以离线建立倒排索引：

$$\text{document} \rightarrow \text{token/term} \rightarrow \text{posting list of document IDs.}$$

此时查询阶段从扫描正文转为查索引，更新文档则要承担分词、词项维护和 `posting list` 更新成本。倒排索引与 KMP 都是 “预处理换查询”，但保存的信息不同：KMP 保存模式自己的 `border`，倒排索引保存语料中的出现位置。不能因为两者都叫 “索引” 就把一个当成另一个的实现。

全文搜索还会引入大小写、分词、规范化、通配符和多模式查询等语义问题；这些属于 `Extension`。408 题目若只给一个文本和一个模式，优先调用本册的单模式契约，不凭 “工业搜索” 背景增加不存在的预处理。

6 概念边界与做题控制协议

概念 A	≠ 概念 B	真正区别	混淆后果
串	≠ 模式	被搜索的文本对象 vs 约束模板	方向写反
border	≠ 任意重复片段	必须同时是前缀和后缀	错误回退位置
prefix function	≠ 匹配答案	模式内部信息 vs 文本中的出现位置	把预处理当搜索
失配回退	≠ 主串回退	只缩短模式状态	复杂度退化
线性时间	≠ 每次比较 O(1)	总比较次数还需证明单调性	复杂度口号化

遇到匹配题，按“固定返回契约 → 写朴素基线 → 找重复前后缀 → 固定 prefix/next 语义 → 画一次失配回退 → 再报告复杂度”的顺序处理。忘掉公式后，只需复原：**已经匹配了什么？失配时哪些后缀仍可能成为模式前缀？模式状态能否缩短而不重扫文本？**

7 全科坐标与来源处理

Map Coordinate: Prefix / Failure / Reuse

DS03 把线性序列上的“匹配失败”转化为模式内部 border 结构；它接收 DS01 的顺序访问成本语言，并向算法专题输出 KMP 的线性扫描接口。自动机、多模式和文本索引是后续 Extension，不改变本册唯一 Owner。

本册吸收归档总册对 KMP 的核心表述“已匹配前缀压缩为失败状态”，并将 next 下标差异明确为表示约定问题。代码和测试只验证本册固定的 prefix-length 语义，避免把不同教材的数组编码混作第二套稳定真相。